Comet: Fine-grained Computation-communication Overlapping for Mixture-of-Experts

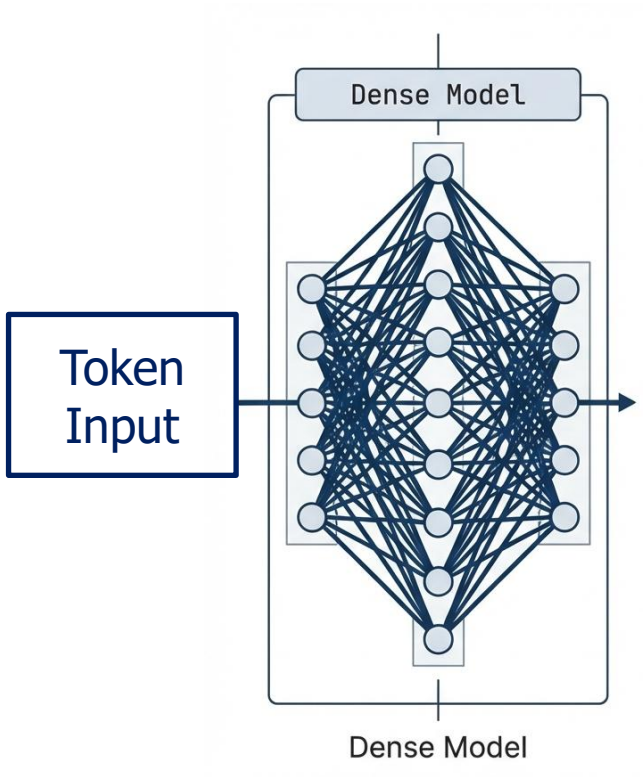
Original Paper by: Shulai Zhang et al. (MLSys '2025, ByteDance)

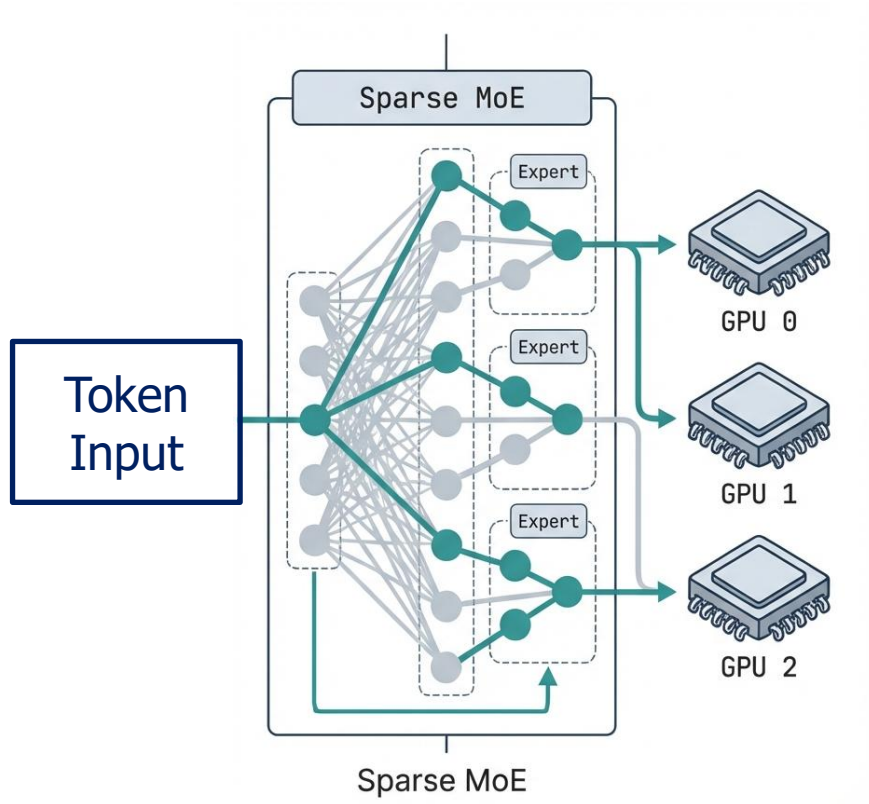
Jongyun Hur

Introduction

Introduction

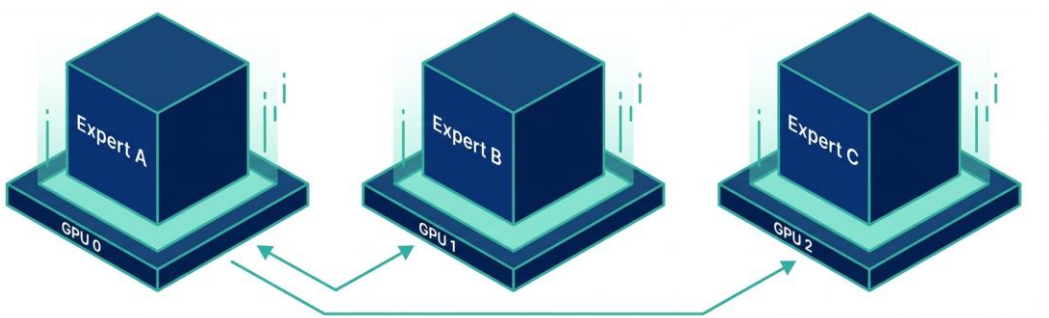
What is MoE (Mixture of Experts)



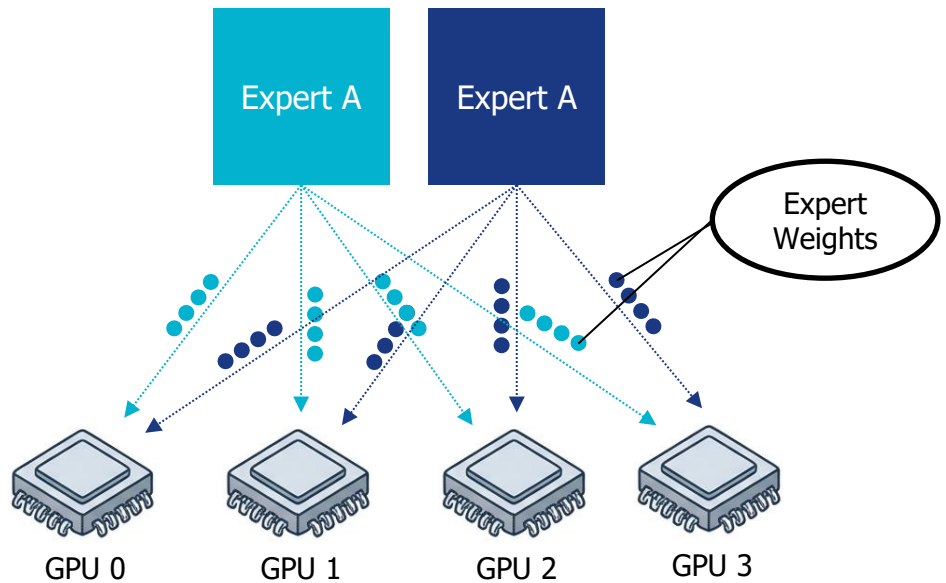


Introduction

What is Expert Parallelism & Tensor Parallelism



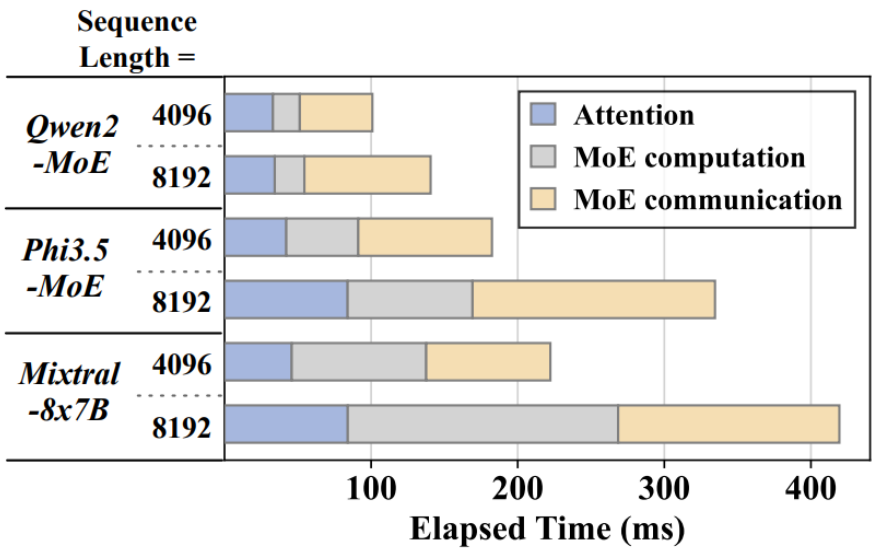
Expert Parallelism (EP)



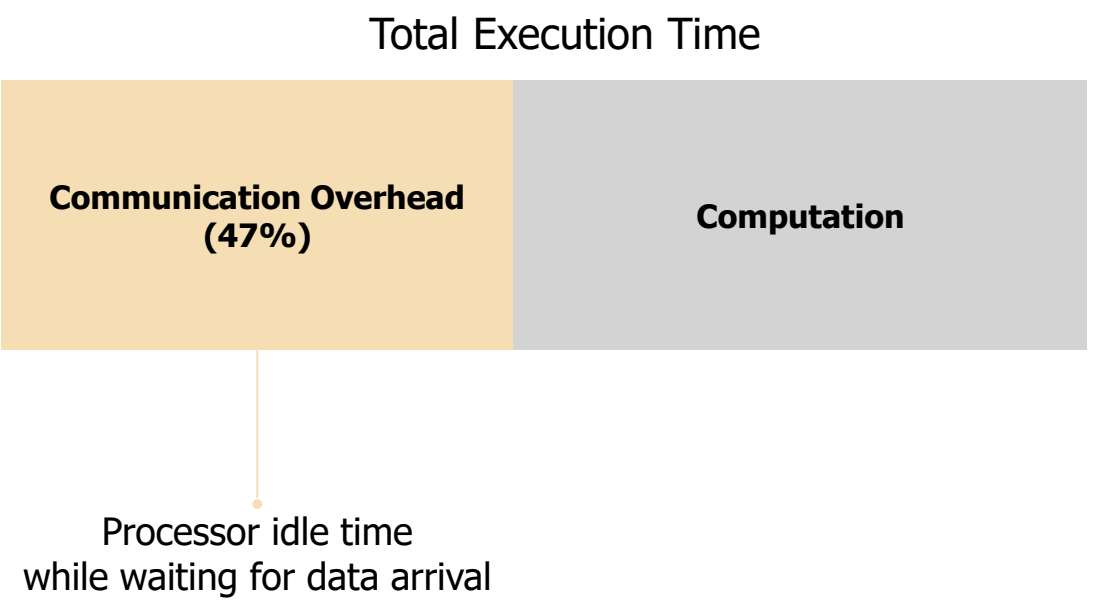
Tensor Parallelism (TP)

Introduction

The communication bottleneck in MoE execution

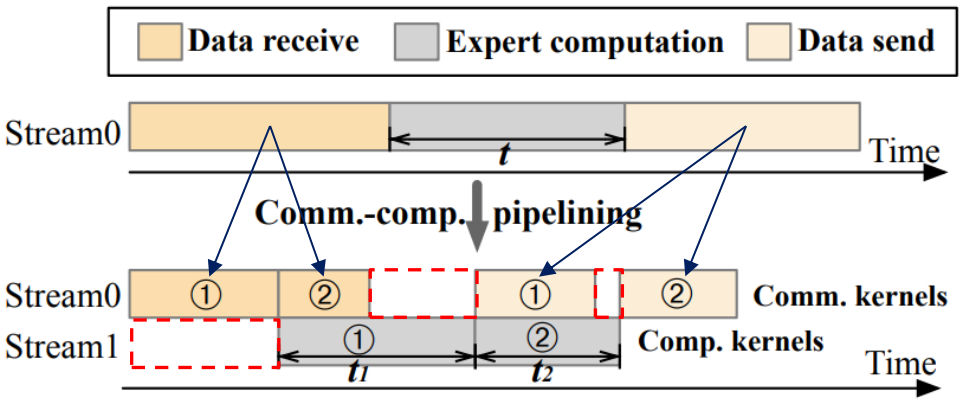


(a) Time breakdown analysis of typical MoE models.



Introduction

The hidden Latency: Communication overhead



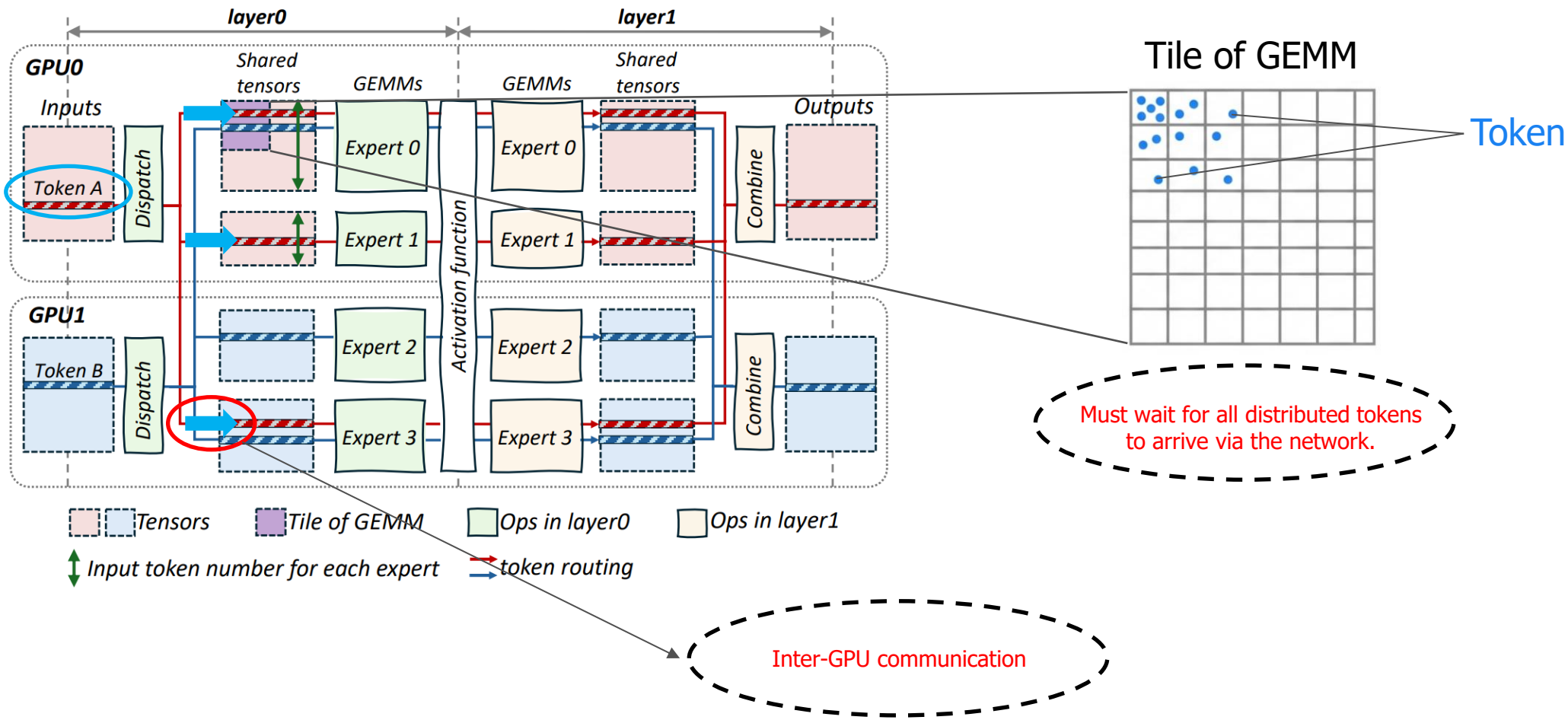
(b) Coarse-grained communication-computation overlap.

Increasing total execution time (t1 + t2 > t)

1. Idle Time (Bubbles): GPU idleness caused by mismatches between communication and computation
2. Efficiency Loss: Small workloads reduce compute efficiency, while overly large ones diminish overlapping benefits.

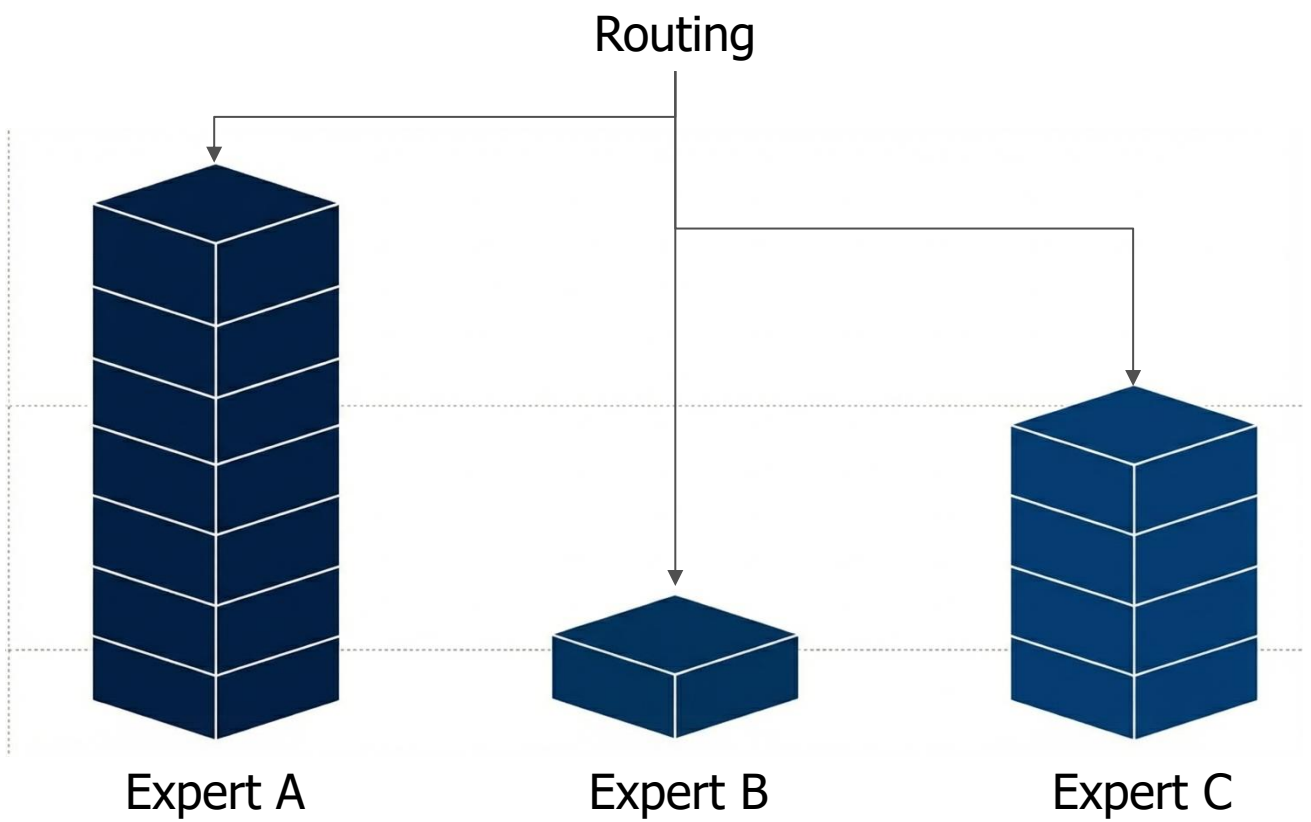
Introduction

The Granularity Mismatch



Introduction

Dynamic Workload Misalignment



Introduction

Why "Coarse-Grained" Solutions Fail

1. Reduced Efficiency: Fragmented kernels lower GPU utilization, increasing total execution time.

2. Structural Idle Time: Inherent bubbles exist at the start and end of communication phases where overlap is impossible.

3. Workload Misalignment: Dynamic workloads and lack of fine-grained control disrupt the timing for seamless overlapping.

Result: Significant processing ‘bubbles’ and GPU idle time.

Contribution

Contribution

Comet: Fine-grained execution for complex dependencies

Innovation 1:
Shared Tensor Based Dependency Resolving

Latency Minimization through Tensor Reordering

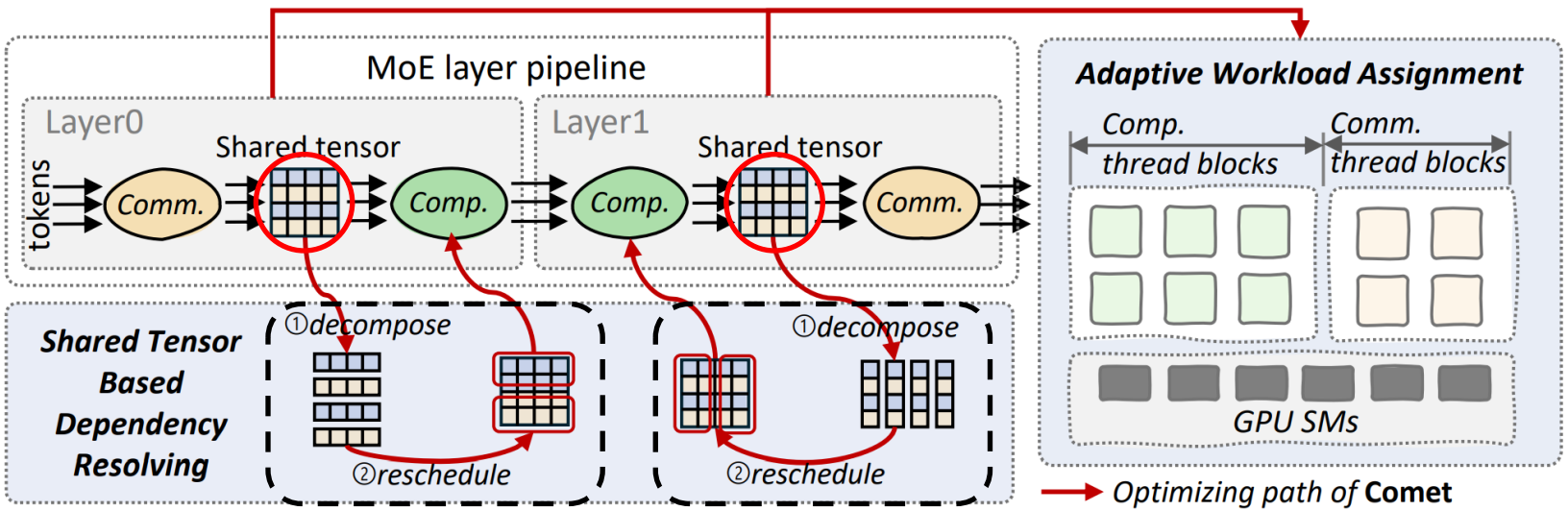
Comet

Innovation 2:
Adaptive Workload Assignment

Isolating Thread Blocks & Dynamic Resource Allocation

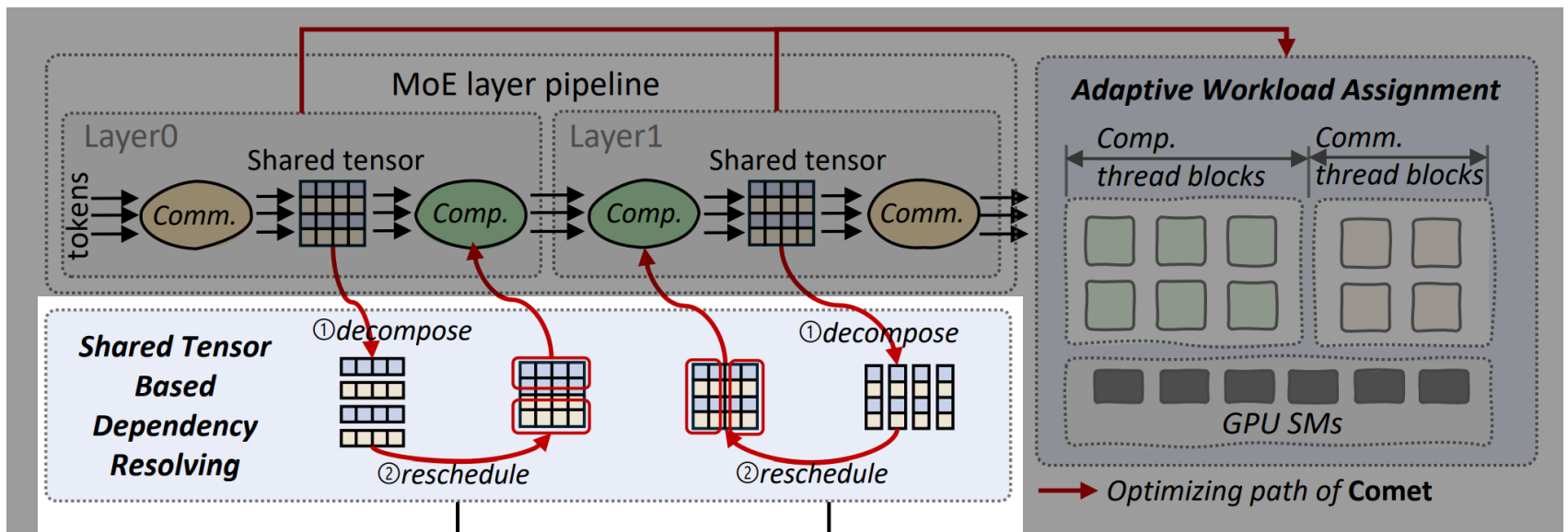
Contribution

Design overview of Comet



Contribution

Design overview of Comet

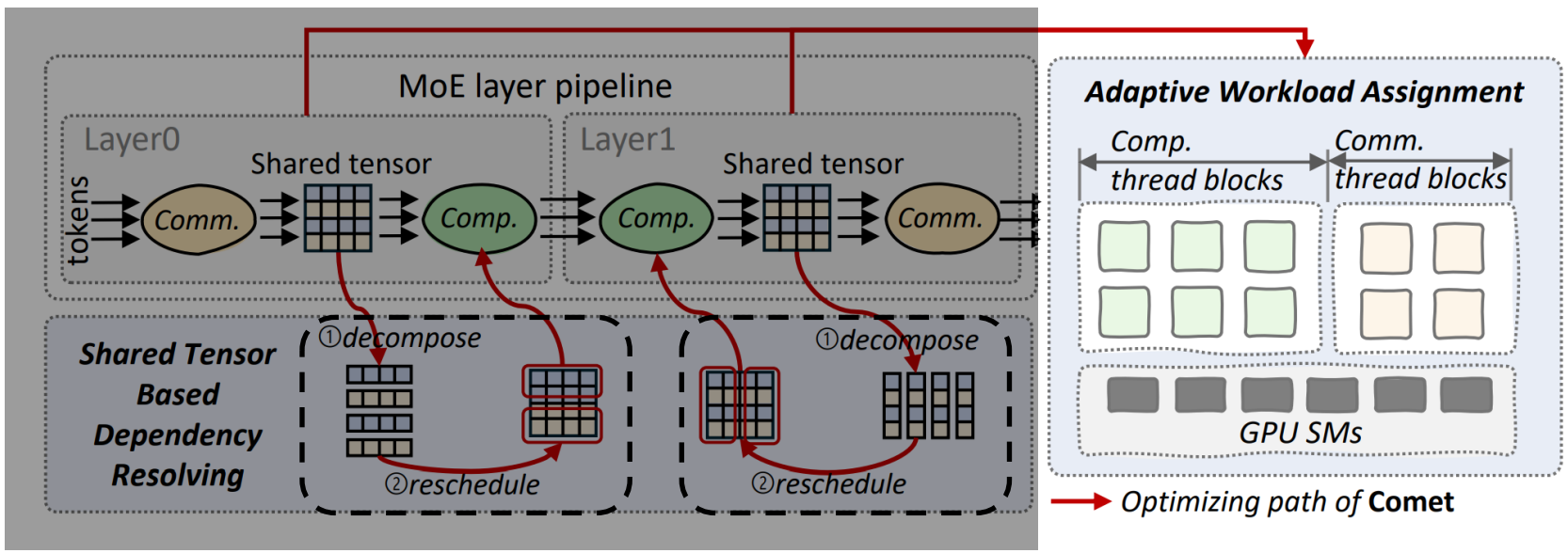


Row-wise Decomposition
Compute immediately upon token arrival.

Column-wise Decomposition
Transmit immediately after attribute computation.

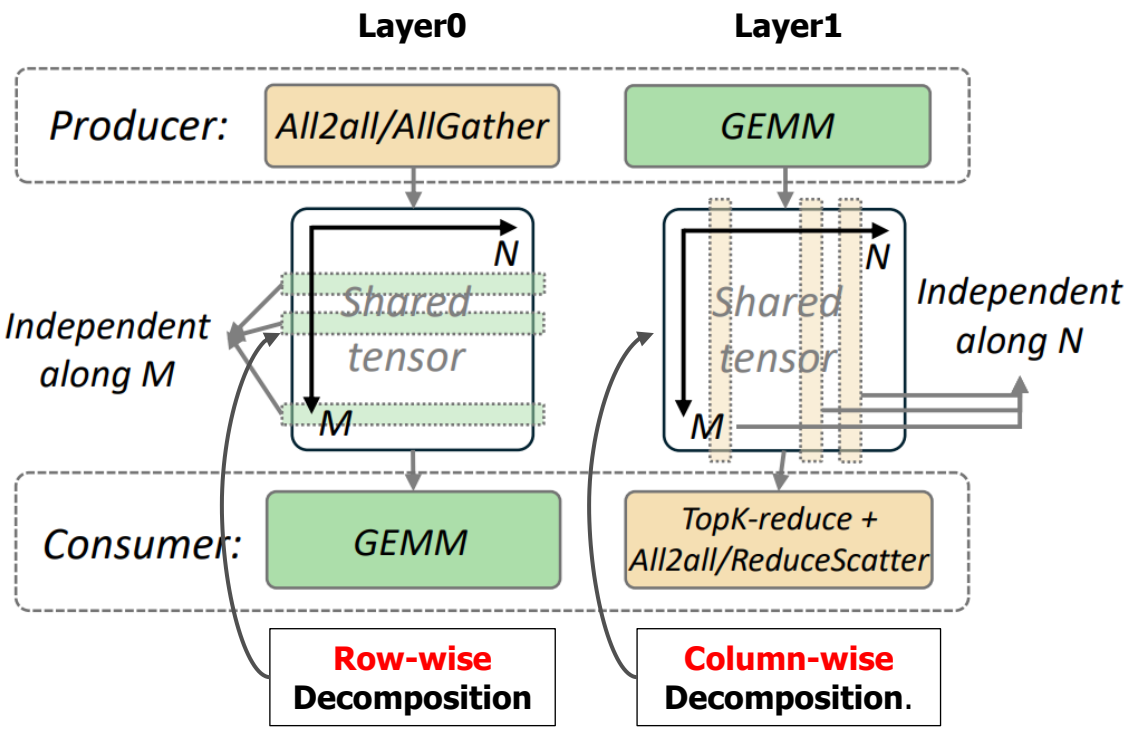
Contribution

Design overview of Comet



Contribution

How to decompose the shared tensor

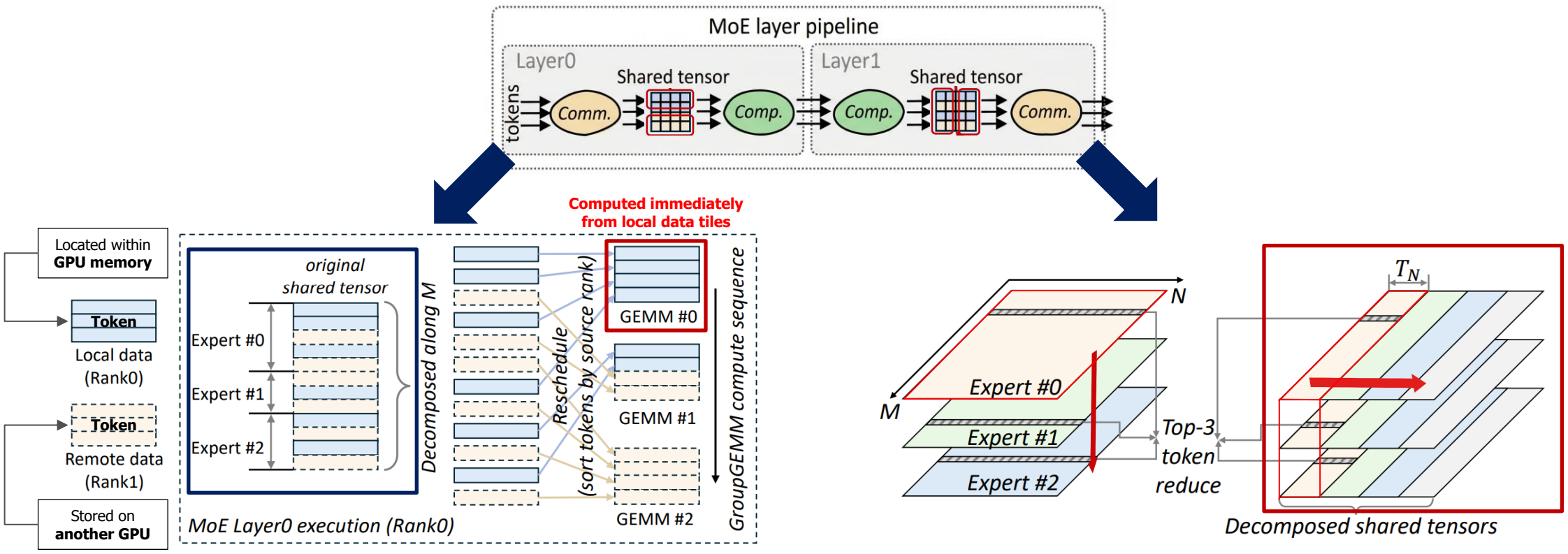


Shared Tensor Example

M (Token Dimension)	N (Embedding Dimension: 4,096 features)
1. I	[0.12, 0.45, 0.91, ... , 0.33]
2. ate	[0.88, 0.11, 0.24, ... , 0.76]
3. lunch	[0.55, 0.67, 0.19, ... , 0.42]
4. today	[0.22, 0.38, 0.84, ... , 0.95]

Contribution

How to reschedule the decomposed shared tensor

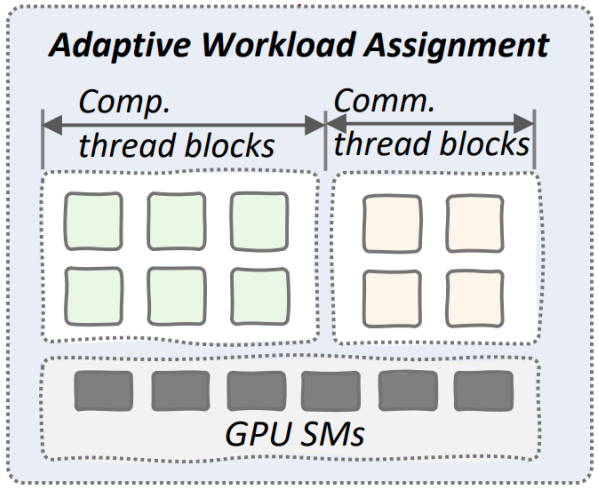


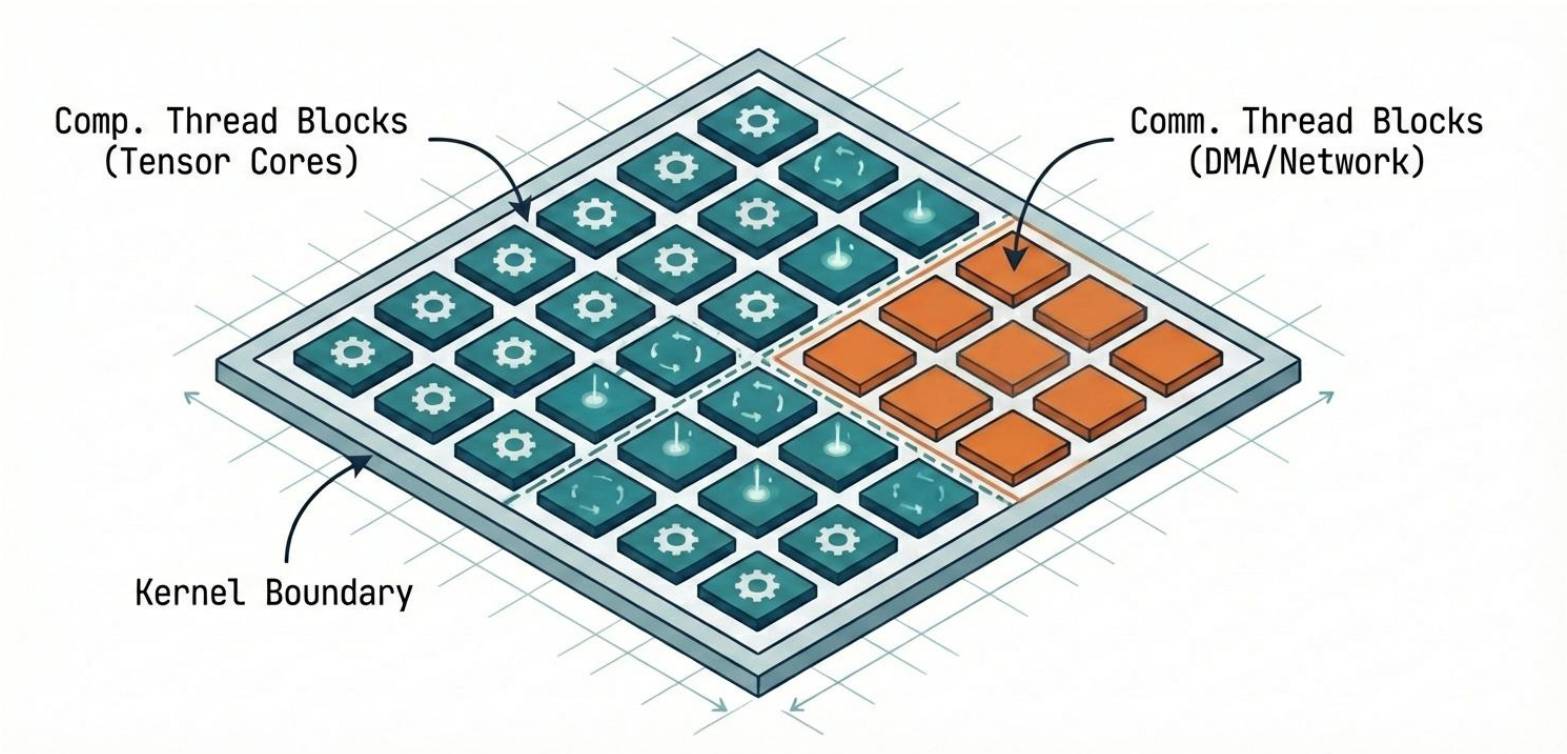
Reschedule operations to process **local tokens first,** overlapping communication with computation to eliminate GPU bubbles.

Layer-by-layer sequential processing → Column-wise concurrent processing across all layers

Contribution

Adaptive thread block assignment

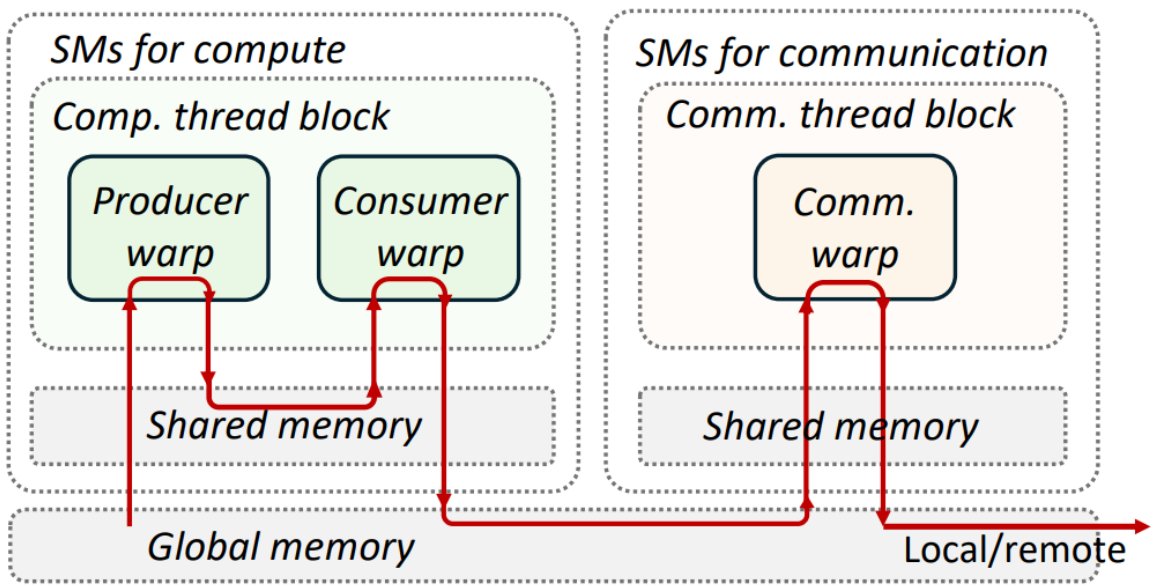




Core Mechanisms

Core Mechanisms

Thread block specialization on Hopper Architecture

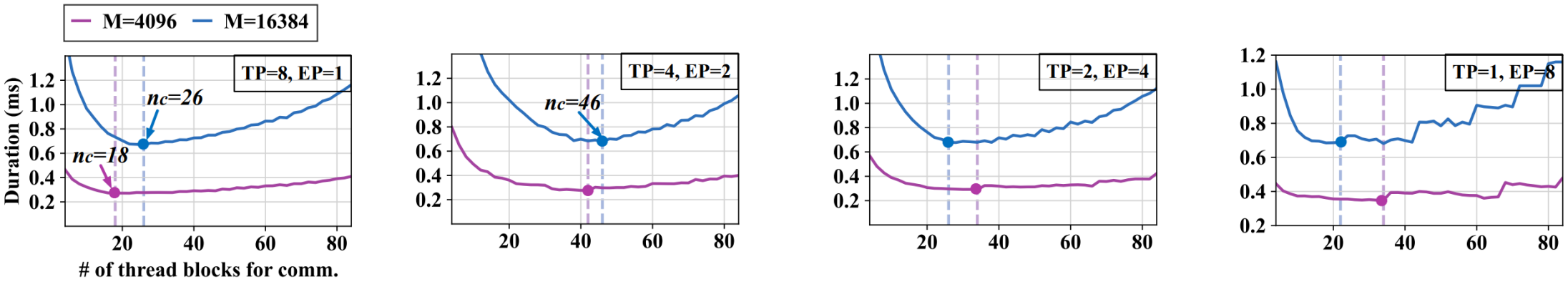


Kernel design for the MoE layer1 on Hopper architecture

Core Mechanisms

Dynamic shifts in optimal resource allocation

n_c	The number of GPU thread blocks dedicated to **communication** within the kernel
TP	**Tensor Parallelism** size, where the weights of experts are partitioned along the hidden dimension across multiple GPUs.
EP	**Expert Parallelism** size, where different experts are distributed intact across separate GPUs



Adaptive workload assignment is essential
as the optimal resource allocation shifts dynamically with varying workloads.

Evaluation

Evaluation

Experimental Setup I: Environment

Hardware Specifications

- Primary Testbed (High-Performance)
  - GPU: 8 × Nvidia H800 (80 GB memory each)
  - Interconnect: NVLink
- Secondary Testbed (Bandwidth-Limited)
  - GPU: 8 × Nvidia L20 (46 GB memory each)
  - Interconnect: PCIe bridges (Bandwidth ≈ 25 GB/s)

Model Name	Transformer layers	Experts	Experts per token	Embedding size of a token	Hidden size of Feed-forward layer
Mixtral 8x7B	32	8	2	4096	14336
Qwen2-MoE-2.7B	24	64	4	2048	1408
Phi-3.5-MoE	32	16	2	4096	6400

Evaluation

Experimental Setup II: Comparison targets

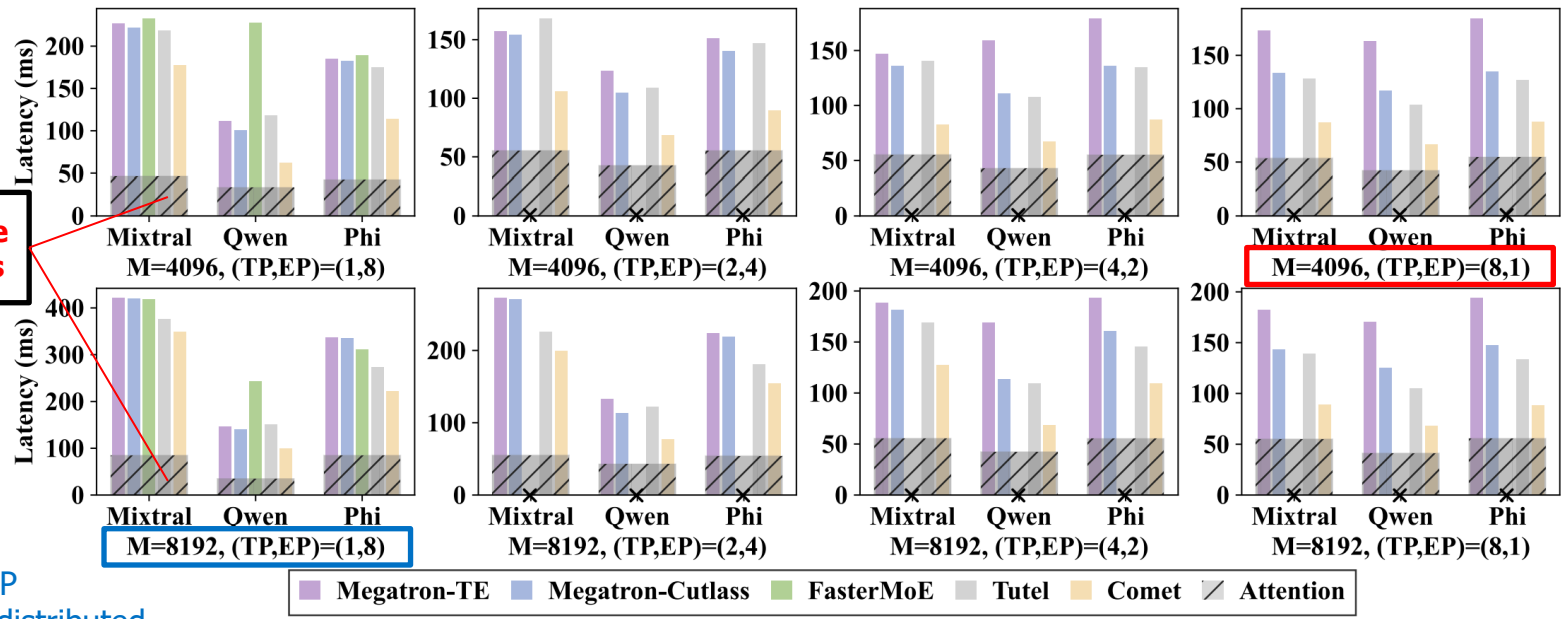
Baseline	Base Framework	Key Technologies & Features	Comm-Comp Overlapping
(a) Megatron-Cutlass	Megatron-LM	• Uses NVIDIA **CUTLASS** library • Implements experts via Grouped GEMM	**Not Supported** (Sequential)
(b) Megatron-TE	Megatron-LM	• Uses NVIDIA **Transformer Engine** • Standard acceleration library for Transformers	**Not Supported** (Sequential)
(c) FasterMoE	PyTorch	• **Customized All-to-All** communication • Pipelined execution of experts	**Supported** (Not supported TP)
(d) Tutel	Microsoft	• **Adaptive Parallelism** • 2D hierarchical All-to-All algorithm	**Supported**

Evaluation

End-to-End performance

Tensor Parallel (FasterMoE restricts the Tensor Parallel size to 1)

The execution time of Attention layers



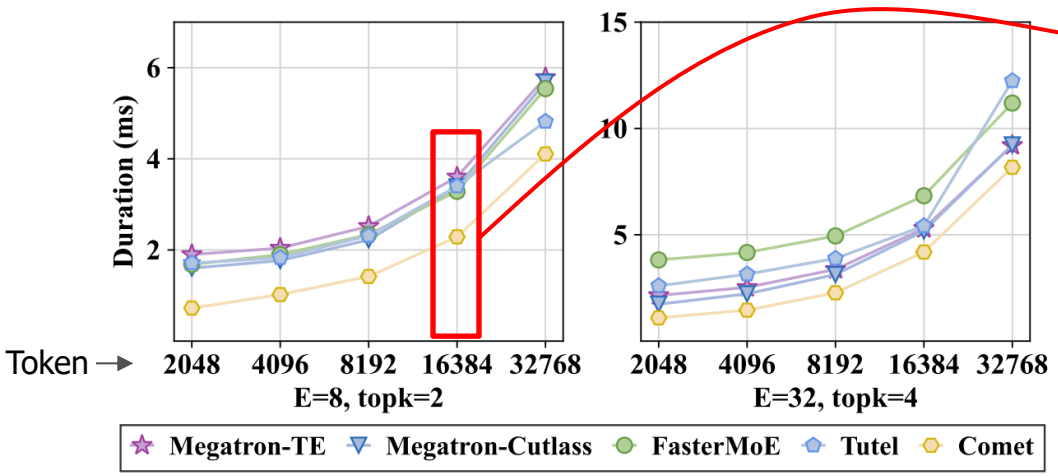
EP > TP
Highly partitioned along the hidden dimension.

TP > EP
Kept intact and distributed as whole units.

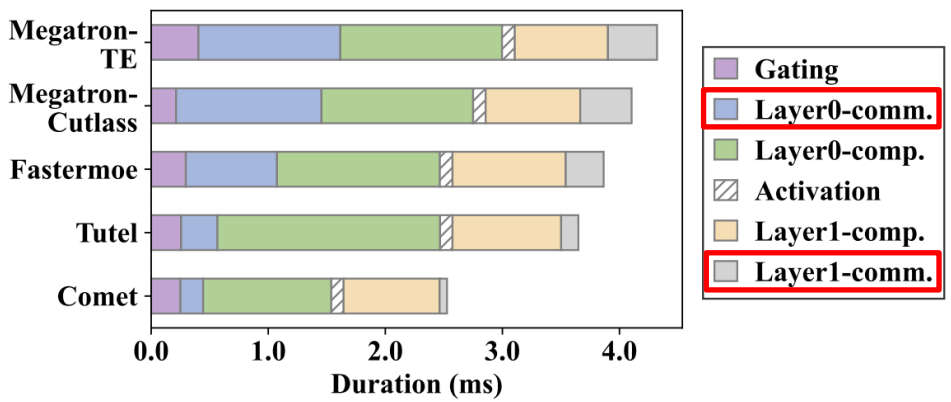
Evaluation

Detailed Evaluation on a Single MoE Layer

Configuration of MoE model: Mixtral 8x7B.



EP=8, TP=1, *E*=8, *topk*=2, Input Length *M*=16384c



Input Scenario (M)	Workload Characteristics	Key Effect
Small Batch (2,048 ~ 4,096)	Communication Time > Computation Time	**Best Speedup** Resolves communication overhead
Large Batch (16,384 ~ 32,768)	Communication Time < Computation Time	**Stable Performance** Provides stable performance enhancement

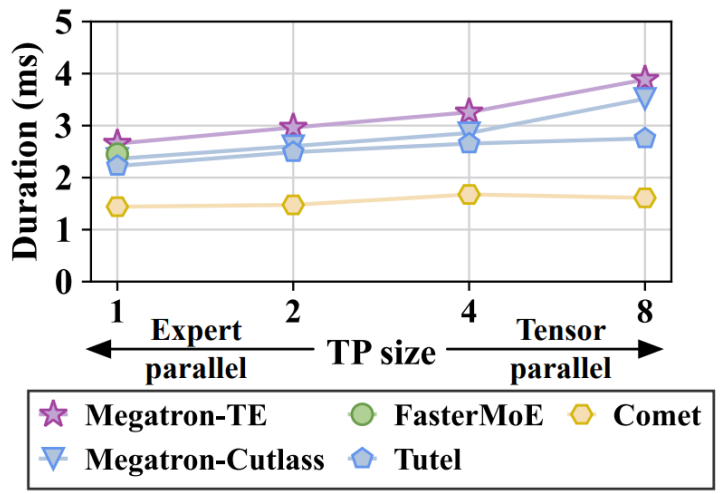
Achieving 86.5% Communication Hiding Ratio with Zero Computational Overhead.

Evaluation

Eliminating efficiency degradation in high Tensor Parallelism

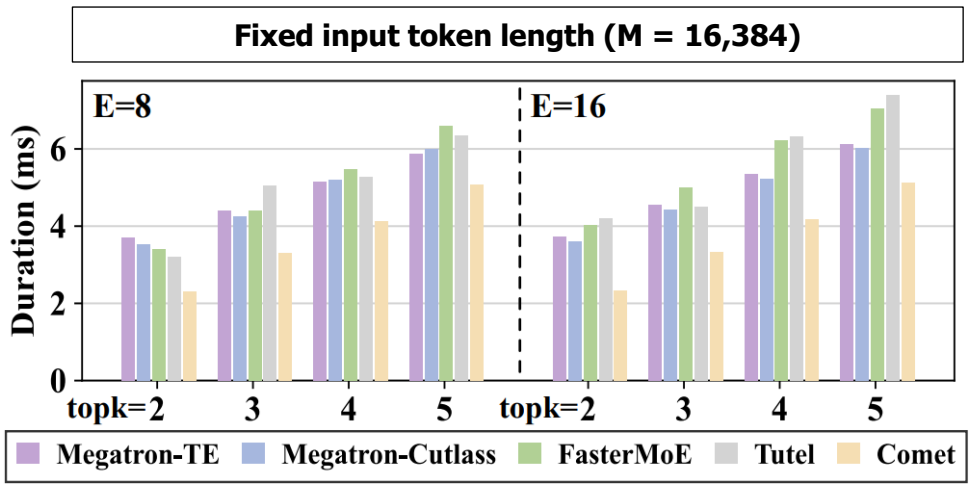
- **Challenge:** Increasing Tensor Parallelism (TP) leads to **fragmented small GEMMs**, degrading computational efficiency in baselines.

- **Comet:** Maintains **consistent high performance** across all parallel strategies by rescheduling shared tensors to eliminate fragmentation overhead.

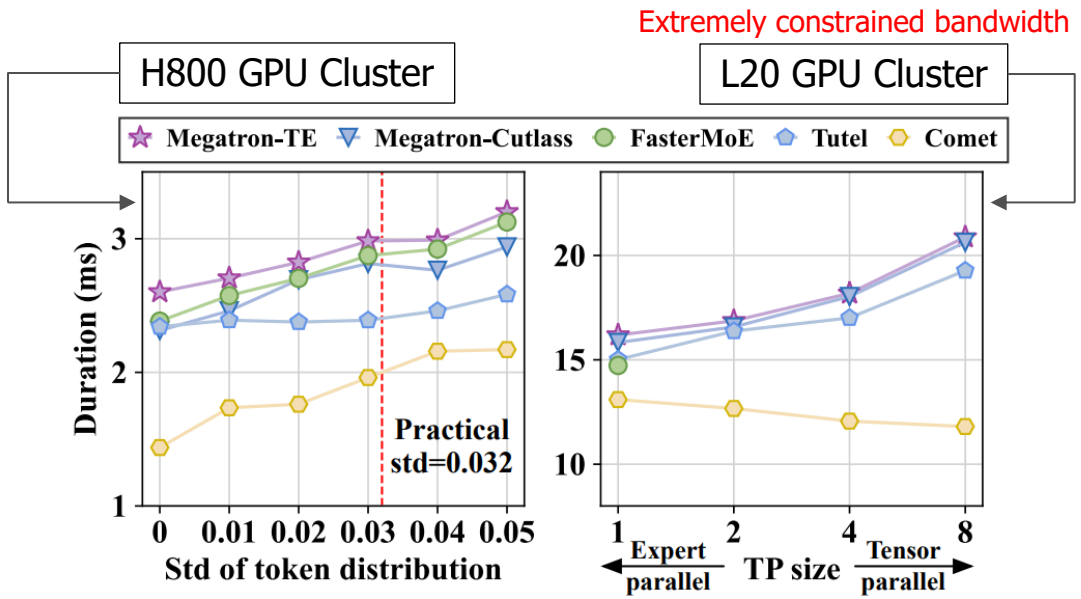


Evaluation

Adaptiveness to Different Configurations



Scalability to Varying Model Hyperparameters (*E*, *topk*)



Robustness against Load Imbalance and Hardware Constraints

Comet delivers superior adaptiveness and performance, even with complex changes in routing hyperparameters.

Increasing Std of token distribution → Extreme workload imbalance

Comet: Lowest latency overhead

L20 GPU Cluster → Constrained bandwidth & PCIe Bridge

Comet: Robust in harsh production

Conclusion

Conclusion

Comet – High performance MoE system

Challenge	Comet's Solution	Benefit
Communication Bottleneck **47%** of total execution time	**Fine-grained Overlapping** Computation-Communication Hiding	**1.71×** **Speedup** End-to-End Acceleration
Complex Dependency GPU idle time & bubbles	**Shared Tensor Rescheduling** Decomposing & Reordering	**Zero Bubble Pipeline** Maximized Utilization
Dynamic Workload Input-varying optima	**Adaptive Assignment** Thread Block Specialization	**Robust Performance** Consistent gains across settings

Comet achieves 1.71× speedup and saves millions of GPU hours via fine-grained overlapping